

Reinforcement-Learned Quadrotor Goal-Reaching in ROS 2 + Gazebo

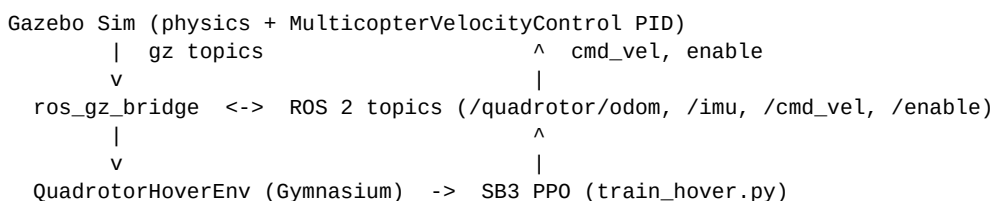
Project report: architecture, history, diagnosis, and redesign
July 15, 2026 • workspace ~/ros2_ws, package quadrotor_sim

Abstract. This project trains a PPO policy to fly a simulated quadrotor to a randomly sampled 3D target from a randomized spawn pose, using ROS 2 Jazzy, Gazebo Sim, and Stable-Baselines3. It began as a fixed-point hover task that the first training run essentially solved, was upgraded into a much harder randomized goal-reaching task, and then spent several training runs stuck on a reward plateau that resisted a long series of individually correct fixes. The root cause turned out to be an environment-level control-timestep bug: each policy step advanced simulated time by only ~10-15 ms instead of the ~50 ms the whole task design assumed, which simultaneously made many targets physically unreachable and destroyed exploration. This document describes the full system, its state before the latest round of changes, the diagnosis, and every change made with its reasoning and verification.

1. The project

The system wires a physics simulator to a standard reinforcement-learning stack. Gazebo Sim (gz-sim 8) simulates a quadrotor whose low-level flight is stabilized by Gazebo's built-in MulticopterVelocityControl plugin: it accepts a body-frame velocity setpoint and runs the motors with a PID loop. The learned policy therefore does *guidance*, not raw motor control: it observes the drone's state and emits velocity commands, and the PID turns those into flight.

ros_gz_bridge translates between Gazebo topics and ROS 2 topics. A custom Gymnasium environment (QuadrotorHoverEnv) subscribes to odometry (50 Hz) and IMU (100 Hz), publishes cmd_vel and an enable signal, and exposes the usual reset()/step() interface. Stable-Baselines3 PPO trains against it. Episode resets teleport the drone via Gazebo's set_pose service over a persistent gz-transport connection rather than deleting and respawning the model.



Key components

Path	Role
quadrotor_sim/envs/quadrotor_hover_env.py	The Gymnasium environment: observation assembly, reward, resets/teleports, crash detection, randomization, curriculum, success termination, control-timestep pacing.
quadrotor_sim/train/train_hover.py	Training entry point: launches headless Gazebo + bridge, builds the wrapped env (Monitor, DummyVecEnv, VecNormalize), configures PPO and callbacks, saves models.

Path	Role
quadrotor_sim/train/callbacks.py	Deterministic best-checkpoint evaluation, periodic checkpoints (with VecNormalize stats), early stopping, and physics pausing during gradient updates.
scripts/eval_hover.py	Success-rate evaluation of a saved model against a live sim (success = final distance < 0.15 m); loads VecNormalize stats and matches training-time step pacing.
scripts/check_sampling_smoke.py	Zero-Gazebo regression suite: sampling ranges, quaternion round-trip, reward/crash bounds, shaping terms, body-frame transform, success-hold, curriculum logic.
worlds/empty.sdf / empty_train.sdf	Demo-fidelity world (1 ms physics, real-time) vs. training world (2 ms physics, uncapped real-time factor).

2. Where it started: fixed-point hover

The original task was to hover at a single fixed point (0, 0, 1.0 m), always spawning from exactly that pose. The observation was a 9-dimensional world-frame vector (position error to the fixed point, velocity, roll/pitch/yaw), actions scaled to a timid ± 0.1 m/s, and the reward penalized distance, vertical speed and tilt. This task was effectively solved: the best checkpoint scored 467.5 out of a theoretical ~ 500 , holding within 0.04-0.11 m of the point.

Solved, however, mostly meant *narrow*. The PID does the actual flying; with a fixed spawn, a fixed target, no noise and no disturbances, the policy only needed to learn tiny corrections around one operating point. That made the project a good foundation but a weak demonstration of learned autonomy, which motivated the upgrade.

3. The upgrade: randomized goal-reaching

The task was generalized so that every episode samples a new 3D target (± 1.5 m xy, 0.6-2.0 m z, at least 0.75 m from spawn) and a new spawn pose (± 0.3 m xy, 0.9-1.1 m z, fully random yaw, small roll/pitch jitter). Several design decisions came with it:

- **Body-frame observation (10-dim).** The position error is rotated into the drone's own heading and yaw is encoded as sin/cos. Because `cmd_vel` is body-frame, this makes the mapping from observed error to corrective action nearly an identity, instead of forcing the network to learn a yaw-dependent rotation. The frame conventions were verified empirically against live Gazebo across all four yaw quadrants.
- **Action scale ± 0.35 m/s.** The original ± 0.1 m/s was sized for millimetre corrections; it left no margin to reach a target up to ~ 2.9 m away within an episode.
- **Absolute crash bounds.** Leaving the arena (z outside [0.05, 3.0] m or $|x|, |y| > 3$ m) terminates the episode with a penalty, independent of where the target is.
- **Legacy mode as a hard contract.** Every new feature defaults to a value that reproduces the original task byte-for-byte (`randomize=False` keeps the 9-dim world-frame observation, the ± 0.1 m/s scale, the fixed spawn/target, the original step pacing). The original hover model still loads and scores its historical numbers, which is re-checked after every change round.

An evaluation harness was added at the same time: `eval_hover.py` reports the success rate over N randomized episodes with success defined as a final distance under 0.15 m.

4. Before the latest changes: the plateau, and everything tried against it

From the first randomized-goal run onward, training showed the same signature: a briefly positive start, a slide to an episode-reward mean around -200, then tens of thousands of steps of oscillation with no net

improvement. Episodes ran their full 500 steps (the drone survived) but the policy never converged on targets — a 100-episode evaluation showed 0% success even at a generous 0.5 m tolerance, plateauing at 0.75-1.5 m from the target. A long sequence of real problems was found and fixed; each fix was individually verified, and none broke the plateau. That history matters, because it ruled out most conventional explanations before the real one was found:

Problem found	Fix applied	Reasoning / verification
Escalating reset timeouts as training progressed	Persistent gz-transport node for pause/unpause/teleport instead of spawning a CLI subprocess per call	Each subprocess created a new discovery participant; churn eventually stalled resets. Verified: 20/20 randomized teleports converge in 1.3-4.5 s.
Critic instability: explained_variance swinging -0.9 to +0.99, value-loss spikes	VecNormalize (obs + reward), Monitor placed inside DummyVecEnv, checkpoint/eval callbacks made VecNormalize-aware	Per-episode return variance is inherently huge when target distance varies 4x. Also fixed a silent bug: the eval callback drove the raw env, feeding the policy unnormalized observations.
No per-step credit for closing distance	Potential-based progress shaping: reward += 50 * (prev_dist - dist)	Ng/Harada/Russell shaping — telescopes to start-minus-end distance, so it cannot change the optimal policy; verified numerically in the zero-Gazebo suite.
No incentive to refine the last half-metre	Gaussian precision bonus (peak 5.0, sigma 0.3 m) + flat success bonus (50) inside 0.15 m	The linear distance penalty gives constant marginal reward per metre everywhere; the 100-episode eval confirmed the policy stopped at 'roughly close'.
Tilt/vertical-speed penalties fight travel	Proximity gate: those penalties fade to ~0 far from the target, full weight near it	A quadrotor cannot move horizontally without tilting; matches a published quadrotor-racing reward-shaping design (fast-far / precise-near).
Crash penalty of -1000 crushed all shaping through normalization	Crash penalty -50 for this task (default -1000 kept for legacy)	The rare -1000 outlier inflated VecNormalize's return std to ~46, dividing every other reward term down to a few percent of its intended weight. Verified live: return std fell to ~20.

State just before the latest round: a 400k-step run with all of the above applied was at ~110k steps and sitting at an episode-reward mean of -219 — statistically indistinguishable from every run before the fixes. Reward design was not the bottleneck.

5. The root cause: a control-timestep bug

The environment's `step()` published the action and then called `executor.spin_once(timeout_sec=0.05)`. The design intent was 'wait 50 ms, then read the new state'. What `spin_once` actually does is return as soon as **one** ROS callback fires. With the IMU publishing at 100 Hz and odometry at 50 Hz, some callback is always ready within ~10 ms — so each policy step advanced simulated time by only ~10-15 ms, roughly 5x less than designed, and often returned on an IMU message while the position observation was still a full odometry cycle stale.

The evidence was in plain sight in the logs: SB3 reported `fps=69` while Gazebo was locked to real time. A true 20 Hz control loop against a real-time simulator can never exceed 20 steps per second; 69 steps per second means each step spans ~14 ms of simulated time. Working through the consequences shows why every reward-side fix was doomed:

- **Many targets were physically unreachable.** A 500-step episode really lasted ~5-7 simulated seconds, not 25. At the 0.35 m/s action ceiling the drone could cover at most ~1.75-2.4 m, while spawn-to-target distances ranged up to ~2.9 m. A large fraction of episodes could not be completed by *any* policy — the ~200 plateau was approximately the ceiling of the physically possible.
- **Exploration was destroyed.** PPO re-samples Gaussian action noise every step. Held for only ~10 ms and low-pass filtered by the velocity PID, that noise averages to near-zero net displacement — a documented failure mode of too-high control frequency in continuous control. The agent almost never wandered into the precision/success bonuses, so the shaping terms added in Section 4 were practically invisible to it.
- **Observations were stale.** Steps that returned on an IMU callback used position data up to 20 ms old relative to the action being chosen, adding avoidable latency to the loop.

Literature review at this point also recalibrated the sample budget: published PPO waypoint-reaching results in comparable simulators take on the order of millions of environment steps for a single environment, with curriculum learning and success-based termination cited as the main levers for cutting that down. The 400k budget had been thin even without the bug.

6. The redesign: changes and reasoning

6.1 Sim-time-paced stepping (the fix)

`step()` now spins the executor until the *odometry sim-time stamp* has advanced by a configurable control period (`control_dt=0.04` s, i.e. 25 Hz, chosen as an exact multiple of the 20 ms odometry period so the loop never silently rounds up to the next sample). This guarantees exactly one control period of simulated time per step at any simulation speed, and guarantees the position observation is fresh. Episodes are genuinely 20 s; maximum travel is ~7 m against a worst-case 2.9 m target distance. `control_dt=None` preserves the legacy spin-once pacing the original hover model was trained against.

```
deadline = odom_stamp + control_dt - 1e-6    # epsilon: float-safe stamp compare
while odom_stamp < deadline:
    spin_once(timeout=0.05)                  # with a 2 s wall-clock guard
```

6.2 Success termination

Holding inside the 0.15 m success radius for 20 consecutive steps (0.8 s) now ends the episode as won. Once the policy gets good this shortens episodes dramatically, concentrating training data on the interesting part of the task, and it directly optimizes the exact criterion the evaluation measures. The environment also reports `is_success` at every episode end, which Stable-Baselines3 picks up automatically and logs as `rollout/success_rate` — a far better progress signal than mean reward.

6.3 Adaptive curriculum

Targets start close (± 0.75 m xy, 0.8-1.4 m z, at least 0.4 m from spawn) and the ranges expand by linear interpolation toward the full task whenever more than 60% of the last 20 episodes succeeded (four levels;

the window clears on each level-up so every assessment uses episodes at the new difficulty). Easy early tasks give the policy reachable successes to learn from before the full difficulty is unlocked — the standard remedy for goal tasks where random exploration rarely touches the goal region. Level-ups are printed to the training log as a live learning indicator.

6.4 A faster, separate training world

With stepping paced by sim time, wall-clock speed became a free variable. Two changes exploit that in a new `empty_train.sdf` used only by training: physics steps of 2 ms instead of 1 ms, and an uncapped real-time factor. Measured effect: ~2.4x wall-clock throughput (about 35-60 environment steps per second), with flight behavior verified identical for this task. The original `empty.sdf` (1 ms, real-time) is kept for demos, the GUI, and legacy evaluation — an explicit regression test showed the legacy model, whose pacing is wall-clock entangled, degrades from 0.106 m to 0.158 m final distance under the training world's settings.

6.5 Physics paused during gradient updates

Gazebo free-runs; during PPO's update phase the drone used to keep flying its last held command with nobody in the loop. At uncapped simulation speed that would mean tens of simulated seconds of unsupervised drift per update. A new callback zeroes the velocity command and pauses physics when a rollout ends, resuming before the next one starts (ordered after the evaluation callback, which still needs a running sim).

6.6 Evaluation matched to training dynamics

`eval_hover.py` gained a `--control-dt` option defaulting to the training value in randomized mode, so evaluated policies act on the same effective dynamics they were trained with; legacy mode always uses the original pacing. The training budget was raised to 1M steps (an overnight run at the measured throughput), with curriculum level-ups and success rate as the signals for reassessing early.

7. Verification

Per the project's working rule — prove the simulation is right before spending hours of training on it — each change was verified at the cheapest level that could falsify it:

Check	Result
Zero-Gazebo regression suite (11 checks): sampling ranges, legacy fixed mode, quaternion round-trip, absolute crash bounds, crash-penalty configurability, progress-shaping telescoping, precision/success bonuses, stabilization gate, body-frame transform, success-hold logic, curriculum logic	All pass
Timestep exactness: odometry stamp advance per step over 200 live steps	min = mean = max = 0.0400 s
Ground-truth solvability: hand-coded P-controller (no RL) over 10 randomized episodes	10/10 reached the target in 58-138 of 500 steps; success-hold termination fired each time
Full production stack burst: PPO + Monitor + VecNormalize + curriculum + eval callback + pause-physics callback across real rollout/update boundaries	Clean run; eval reward already positive (+120) on the easy curriculum after ~500 steps — no previous full run ever reached positive eval reward
Legacy regression: original hover model on the original world after all changes	100% success, 0.090-0.120 m final distance — matches historical numbers
Throughput: training world vs. original	~2.4x wall-clock (58.8 steps/s free-running)

The decisive item is the P-controller test: it separates 'the task is impossible' from 'the learner is failing'. Before the timestep fix an equivalent controller could not have reached distant targets within an episode; after it, a three-line proportional law solves the task with a wide margin, so PPO is now being asked a fair

question.

8. Status and outlook

All changes are implemented, tested, and documented in the repository README; the previous run's checkpoints are preserved (checkpoints_goal_pre_timestep_fix/), and the 1M-step training run is ready to launch. Early health signals: rollout/success_rate appearing and climbing, mean episode length dropping below 500 as success termination engages, and curriculum level-up lines in the log. Future work already identified: per-episode mass/inertia randomization, wind disturbances, IMU-level sensor noise, camera/lidar perception, multi-waypoint trajectories, and an algorithm comparison (SAC/TD3).

Key references: Ng, Harada & Russell (potential-based reward shaping); action-persistence / control-frequency analyses in continuous-control RL (arXiv 2001.00449, 2010.04304); curriculum RL for quadrotor tasks (arXiv 2501.18490); reward design and termination conditions for RL quadrotor waypoint control (arXiv 2605.19166); gym-pybullet-drones (arXiv 2103.02142).